



22 июля 2026 года · 10 минут чтения

Поделиться

После того как союзники взломали шифр «Энигма» во время Второй мировой войны, они столкнулись с необычной проблемой. Труднее всего было не расшифровать немецкие сообщения. А решить, как использовать полученную информацию, не раскрывая, что она у них есть.

Если бы каждый конвой внезапно стал избегать встречи с каждой подводной лодкой, а каждый авианалет был бы перехвачен, немцы в конце концов поняли бы, что «Энигма» скомпрометирована. Иногда разведданные намеренно не использовались, потому что сохранение тайны было важнее, чем предотвращение одной атаки.

В последние несколько дней я размышлял о том, не сталкивается ли безопасность приложений с открытым исходным кодом с подобной проблемой.

Несколько месяцев назад я писал, что [у защитников наконец-то появилось преимущество](#). ИИ помогает разработчикам понимать отчеты об уязвимостях, проверять код, писать патчи и развертывать исправления гораздо быстрее, чем раньше. Защитники могут каждую неделю улучшать свою кодовую базу и регулярно прерывать цепочки эксплойтов, в то время как злоумышленникам по-прежнему нужно разобраться в уязвимости, разработать эксплойт и найти цели, которые не обновлялись.

Я по-прежнему в это верю. Изменились возможности злоумышленников.

Благодаря искусственному интеллекту стало гораздо проще выявлять уязвимости и превращать небольшие фрагменты общедоступной информации в работающие эксплойты. Время, которое проходит с момента исправления уязвимости до того, как кто-то другой ее воспроизведет, стремительно сокращается.

В течение многих лет публикация патча обычно давала защитникам некоторое преимущество во времени. Кто-то должен был выявить изменения, влияющие на безопасность, понять, почему изменился код, реконструировать исходную уязвимость и создать работающий эксплойт. Для этого требовались опыт, упорство и достаточная мотивация, чтобы часами читать код.

Сегодня альтернативные издержки, связанные с попытками, практически равны нулю. Поиск нужного коммита занимает минуты. Чтобы попросить большую языковую модель объяснить суть изменения, нужны секунды. Если первое объяснение неверно, повторная попытка почти ничего не стоит.

За последние несколько дней мы наблюдали это дважды.

В случае с [Fastjson RCE](#) исследователи публично упомянули об уязвимости, не раскрывая всех технических подробностей. Этого сигнала было достаточно, чтобы другие люди начали расследование, и рабочие эксплойты появились еще до публикации полного описания.

Недавняя уязвимость WordPress wp2shell развивалась по схожему сценарию. Еще до того, как исследователи опубликовали свое исследование, другие люди уже восстановили уязвимость и разработали рабочие эксплойты.

В статье Spaceraccoon о ["Negative Days"](#) та же проблема рассматривается с другой стороны. Как только патч попадает в публичный репозиторий, он сам по себе становится рекомендацией. Злоумышленникам не нужно описание CVE или подробная статья. Они могут сравнить релиз с предыдущей версией, выявить интересные изменения и обнаружить уязвимость до того, как многие защитники безопасности установят исправление. Производители браузеров уже много лет знают об этой проблеме, которая называется «пробелы в патчах». Теперь большие языковые модели распространяют ее на все публичные репозитории.

И в случае с Fastjson, и в случае с WordPress не было необходимости в полной информации. Достаточно было знать, что существует что-то интересное.

Заявить в соцсетях, что у вас есть рабочий эксплойт, — это все равно что Черчилль, выступающий по радио и объявляющий о взломе «Энигмы». Вы не делитесь подробностями, но сообщаете всем, что есть что-то, что стоит найти, и примерно указываете, где искать. Блетчли-Парк хранил свой секрет десятилетиями: о взломе «Энигмы» стало известно только в 1970-х годах, почти через тридцать лет после окончания войны. В наши дни эксплойты не переживают тизерного твита.

None of this is completely new.

The Linux kernel was criticised for years for quietly fixing security bugs without calling them security fixes. No CVE, no advisory, just a small commit with an innocuous message. The official position was that a security fix is just another fix. Whatever you think of that argument, the effect was the same: the patch shipped without telling attackers where to look.

Maintainers have tried to make security patches harder to reverse engineer for years. If you have spent enough time reading patches, you have probably seen security fixes mixed with unrelated refactoring, formatting changes, variable renaming or large amounts of otherwise unnecessary code movement.

The goal was not to hide the vulnerability forever. It was to increase the cost of understanding the fix and give defenders more time.

That approach worked reasonably well because humans have limited attention. If understanding a vulnerability required reading through a few thousand changed lines, many people decided it was not worth the effort.

I am not convinced that assumption still holds.

LLMs do not get tired of reading diffs. They do not care whether the relevant change is surrounded by twenty lines of code or two thousand. They are increasingly good at filtering out noise, comparing behaviour and identifying the handful of changes that matter.

Making the haystack bigger no longer makes the needle much harder to find.

Security patches are usually judged using traditional criteria. Is the patch correct? Does it completely fix the issue? Is it easy to review? Is it easy to backport?

Those are all sensible goals, but a clean and minimal patch can also provide an attacker with an unusually precise description of the vulnerability.

Perhaps we need to ask another question:

How much intelligence does this patch reveal?

This is not an argument for security through obscurity. Open source depends on transparency. Users need timely patches, maintainers need to collaborate, and responsible disclosure still matters.

But the information contained in a patch has become part of the attack surface.

Maintainers may need to think beyond fixing the exact line where a vulnerability appears. Removing an exploit primitive may reveal less than applying a narrow fix. Preparatory refactoring could happen before a security release, rather than inside it. Projects may need to consider whether a patch is safe to publish before most users have had a realistic opportunity to deploy it. Bletchley Park invented reconnaissance planes to explain away its intelligence. Maintainers may need their own cover stories: fixes that ship as optimisations, refactoring or routine cleanups. Adding noise to a diff no longer works, but misdirection about why the code changed still might.

I do not know what the right solution is, or whether open source projects can meaningfully solve this without sacrificing the transparency that makes them open source in the first place.

What I do know is that the economics have changed.

Reverse engineering a security patch used to require time, skill and motivation. Increasingly, it requires curiosity, a public repository and access to an LLM.

Bletchley Park understood that intelligence has its greatest value before the adversary knows what you know.

Open source maintainers may now have to think about the same problem.



Louis Nyffenegger

FOUNDER AND CEO @PENTESTERLAB



RELATED POSTS

When Code Is Cheap, What Happens to AppSec?

Defenders Finally Have the Edge

CVE in Your CVE: When the Patch Doesn't Fix the Vulnerability

Security conferences in the age of AI



Master advanced web hacking and security code review with hands-on labs.

LEARN

[Go Pro](#)

[Exercises](#)

[Bootcamp](#)

[AppSecSchool](#)

[Live Training](#)

COMPANY

[About](#)

[Brand Assets](#)

[Privacy Policy](#)

[Terms of Service](#)

RESOURCES

[Blog](#)

[Security Glossary](#)

[The Field Guide \(Book\)](#)

[Enterprise](#)

[Merch Store](#)

SUPPORT

[Contact Us](#)

[Discord Community](#)

